Aqsa Khalid
Lecturer
School of Computing

CS3002 Information Security



User Authentication

Authentication



- Verifying the identity of the user connecting to a system

- Once authenticated, system will proceed to check **authorization**
  - what resources user is allowed to access
  - what privileges they have

Means of Authentication



I. **Something you know (knowledge)**
   - password, pin, security Q/A

II. **Something you have (possession)**
   - mobile number, token (code generator), smartcard

III. **Someone you are (biometrics)**
   - fingerprint, facial features, voice

IV. **Something you do (behaviour)**
   - typing rhythm, computer usage pattern, mouse movement, touch interaction, handwriting

- **Multifactor authentication**: combining more than one modes from the above list

Authentication Types



- **Repudiable Authentication**
  - involves factors, “what you know” and “what you have,”
  - the information presented can be unreliable because such factors suffer from several well-known problems
  - e.g. passwords can be leaked, possessions can be lost, forged, or easily duplicated.

- **Non-repudiable Authentication**
  - involves characteristics whose proof of origin cannot be denied.
  - includes biometrics like iris patterns, retinal images, hand geometry
  - they positively verify the identity of the individual.

Implementing Authentication



- **Basic authentication involving a server**
  - server maintains a file of usernames and passwords (or some other authenticating information)
  - client sends this information, which is examined by server and if correct, authorization is granted.
- **Challenge-response**
  - the server or any other authenticating system generates a challenge to the host requesting for authentication and expects a correct response.
- **Centralized authentication**
  - a central server authenticates users on the network and in addition also authorizes and audits them.

Password-based Authentication

(something you know)



Password-based Authentication



- secret = user's password

- **User:** provides their identity uid and proof (password)
- **System:** verifies the proof
  - case #1:
    - system knows all user's secrets in cleartext (!!!)
    - to check: proof == $secret_{uid}$?
  - case #2: use one-way hash (digest)
    - system knows the digests of all user's secrets
    - to check: hash(proof) == $digest_{uid}$?

Passwords Pros



For users

- Easy to remember (but if only for one system)

For system administrators

- Easiest to implement, compared to other auth methods
- Users are familiar with the concept, do not require training
- Lowest cost: no specialized hardware needed

Passwords Cons



For users

- can't remember too many passwords
  - either use same password for multiple services !!!
  - or use simple easy to remember passwords
    - but that makes them guessable (son's name!)
  - or use some form of password storage
    - post-it notes !!!
    - client-side password manager app (aka vault)
- vulnerable to shoulder surfing, phishing, social engineering

Passwords Cons



For system administrators

- password files are very frequently a stealing target for hackers. So, server-side password storage remains a challenge
  - hashing with a strong hash function is must
  - even hashed passwords are vulnerable to dictionary attacks

- passwords are readable during transmission
  - encrypted channel is a must

Offline Password Cracking



- When hackers manage to steal the **password file** from a service's database, they immediately get a big dataset of hashed passwords.

- Since hash is a one-way function, attacker needs guesswork and a lot of computation to work out the actual passwords.
  1. make a guess g_i
  2. compute $h_i = \text{hash}(g_i)$
  3. search h_i in the file. If found, a hash has been cracked, g_i is the revealed password.
  4. loop back to step 1 until all hashes cracked, or until a set time.

Offline Password Cracking



- A naive way of guessing is to **brute-force**.
  - first guess or assume the length of target password, say n
  - then try every possible n-character combination of letters, numbers, and symbols
  - quickly becomes infeasible when target passwords are longer

- Other type of offline cracking is **dictionary attack**, where the password guesses are drawn from a list of commonly-used or previously-leaked passwords.
  - Such a list is called *dictionary*
  - These attacks require far less computing effort than brute forcing, but will only reveal passwords present in the dictionary.

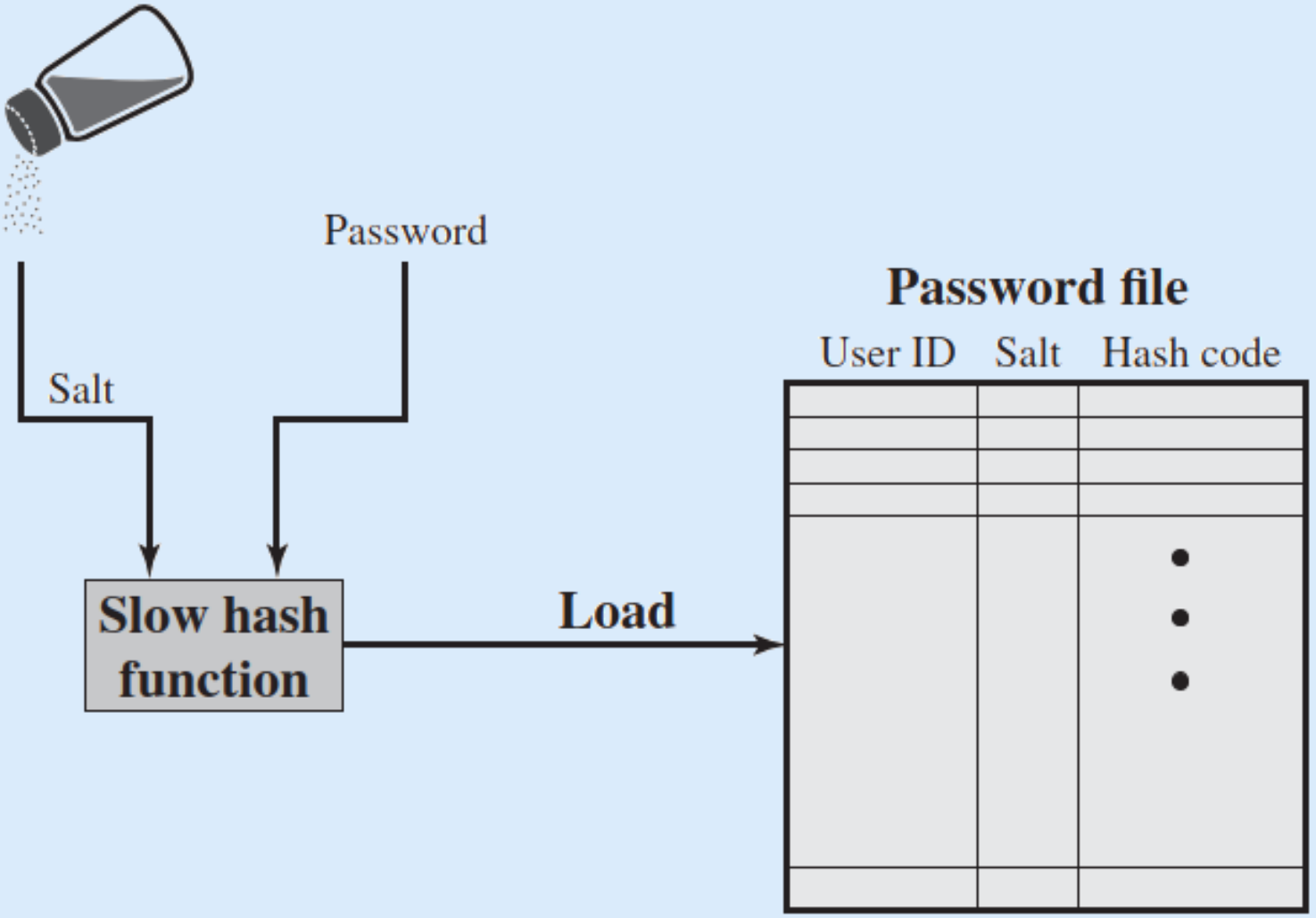
Passwords: Salted storage



- To thwart dictionary attacks, passwords can be **salted**.
- At registration time, for each user UID:
  - create / ask for the password
  - generate a salt – a new random value for each user
- compute HP = hash (password | salt)
  - i.e. concatenate password with salt before hashing
- store the triples { UID, HP, $salt_{UID}$ } in file

Passwords: Salted storage

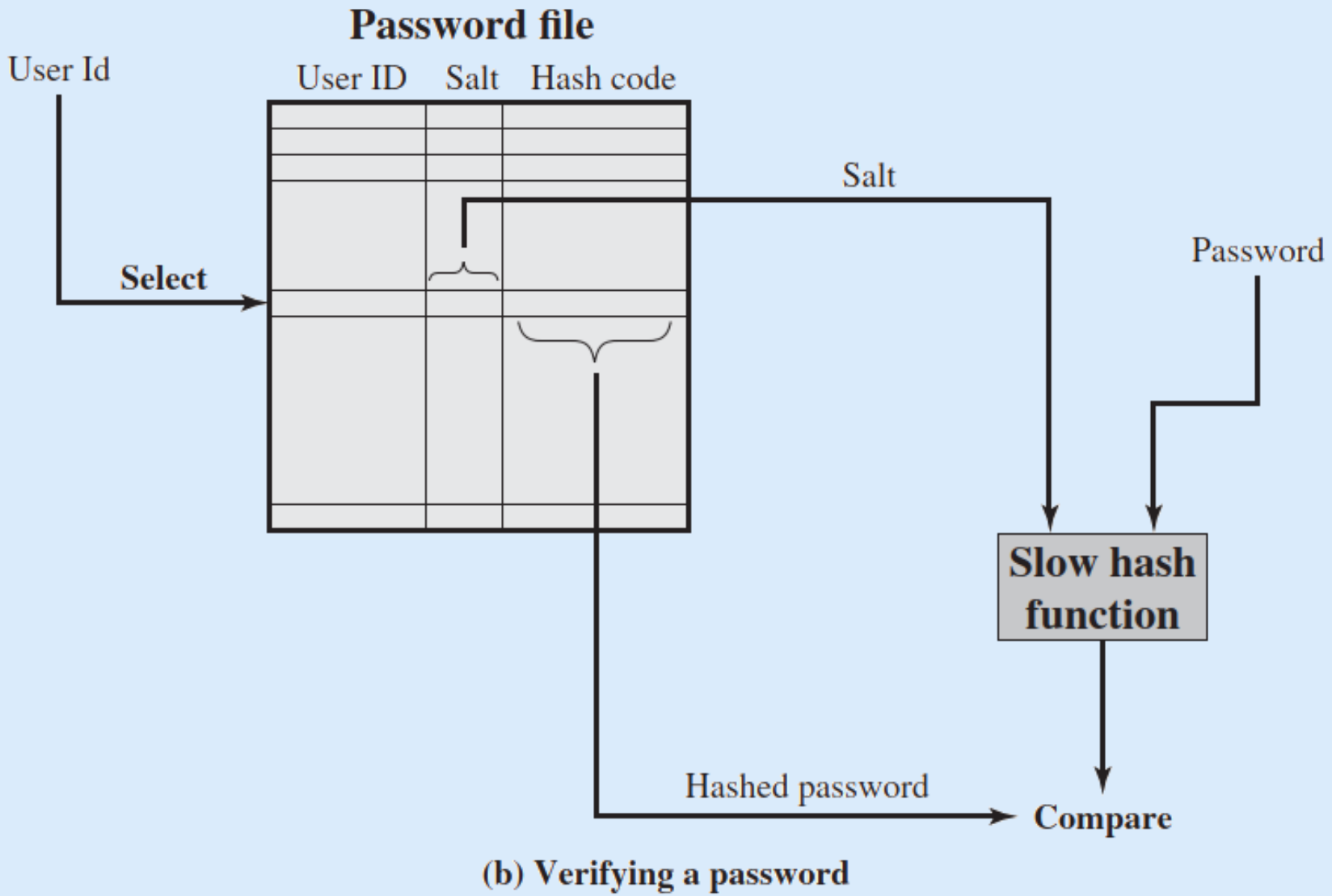




(a) Loading a new password

Passwords: Salted storage





(b) Verifying a password

Advantages of Salting



- Prevents duplicate passwords from being visible in the password file (different HP for users having the same password).
- Significantly increases the time complexity of offline brute force and dictionary attacks, since salt is unique for each user.
  - Attacker will need to crack salted passwords one at a time, whereas previously they could search the whole password file for each guess.
- Makes the **rainbow tables** completely ineffective.
  - A rainbow table is a pre-computed list of plaintext passwords and their corresponding hash values. Without a salt, attacker can quickly look up the stolen hashes in those tables.

Remote User Authentication



- Local authentication is safer
  - Login to personal computer
  - PIN input at ATM machine

- Remote authentication problem
  - Eavesdroppers listening, looking for passwords
  - Solutions:

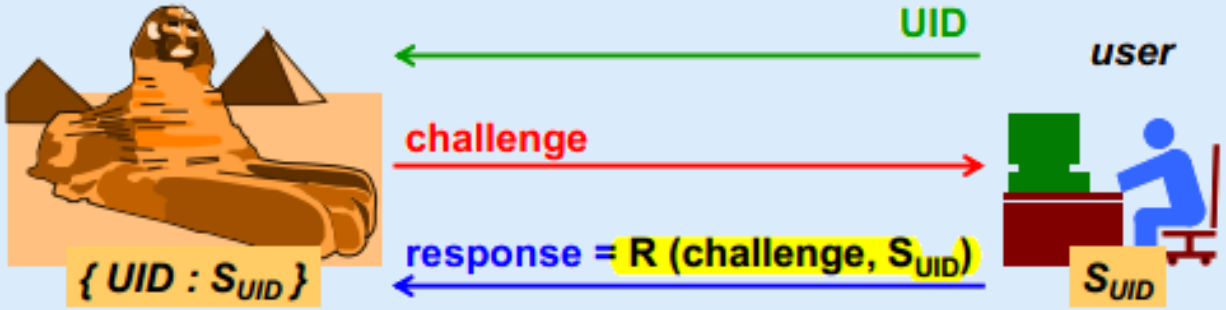
  1) transmit password over an encrypted channel
     - i.e. manage the overhead of creating a secure channel first
  2) use a challenge-response strategy

Challenge-response authentication



Using Symmetric Crypto

- server sends a challenge (a nonce) to the user …
- … who replies with the solution after a computation involving the shared secret and the challenge
  - e.g. encrypt or HMAC the nonce using the shared secret as key
  - the server should know the secret in clear!
- $R()$ is a non-secret mathematical function



Possession-based authentication

(something you have)



Possession-based authentication



- Possession-based authentication verifies a user's identity using something they physically possess, such as a smartphone, hardware token, smart card, or security key.
- Since there is a real risk of that physical object being stolen, possession-based authentication is typically used as **a second factor** of authentication (rather than the only factor).
- A common way to implement it is via **One-Time Passcodes (OTP)**.

One-Time Passcodes (OTP)



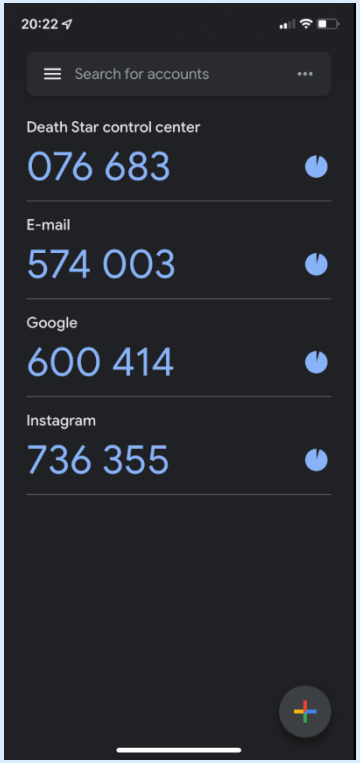
- An OTP is generated from a **secret (seed).**
- The secret could be held by server only.
  - OTP is generated by server and sent to client's second factor device (like mobile phone via call/sms)
- OR the secret could be shared between client and server. So, the server verifies the OTP created at client side
  - generated and shown by a hardware token device
  - generated and shown by an app on smartphone
  - emitted by smart card or hardware keys (connected over USB, Bluetooth or NFC)



Hardware token



YubiKey



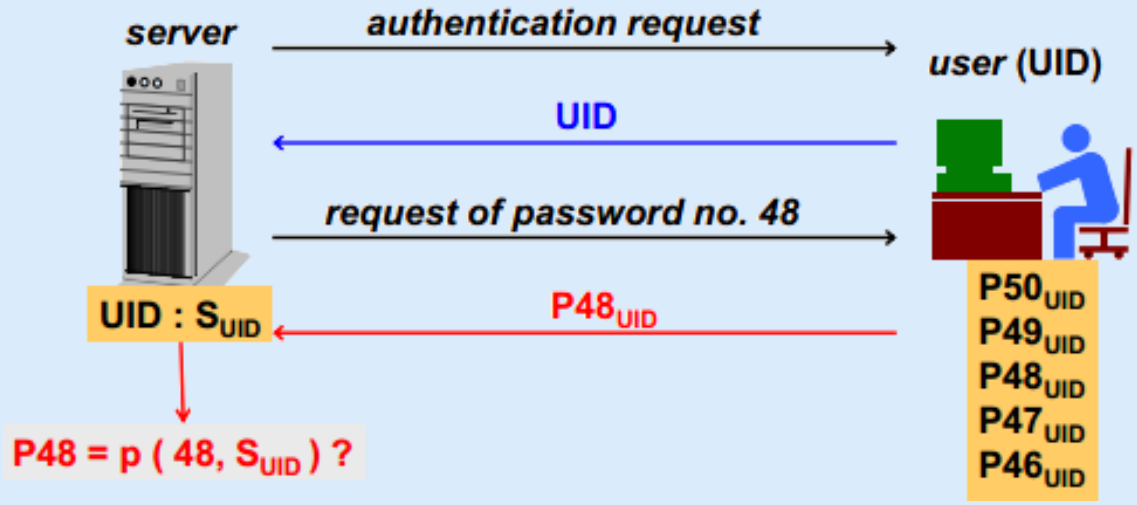
Code generator app (google authenticator)

One-Time Passcodes (OTP)



Counter based one-time passcodes

- A secret value (seed) is securely handed over to client
- At authentication time, OTP is calculated at the client using **an increasing counter** and seed
- A one-way function is used for OTP calculation (why?)

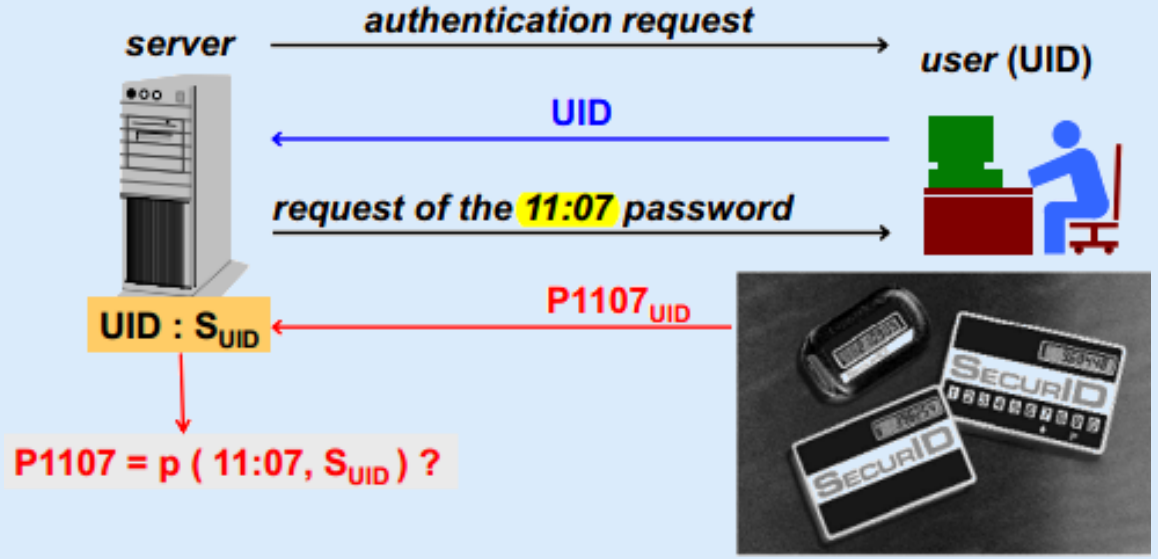


One-Time Passcodes (OTP)



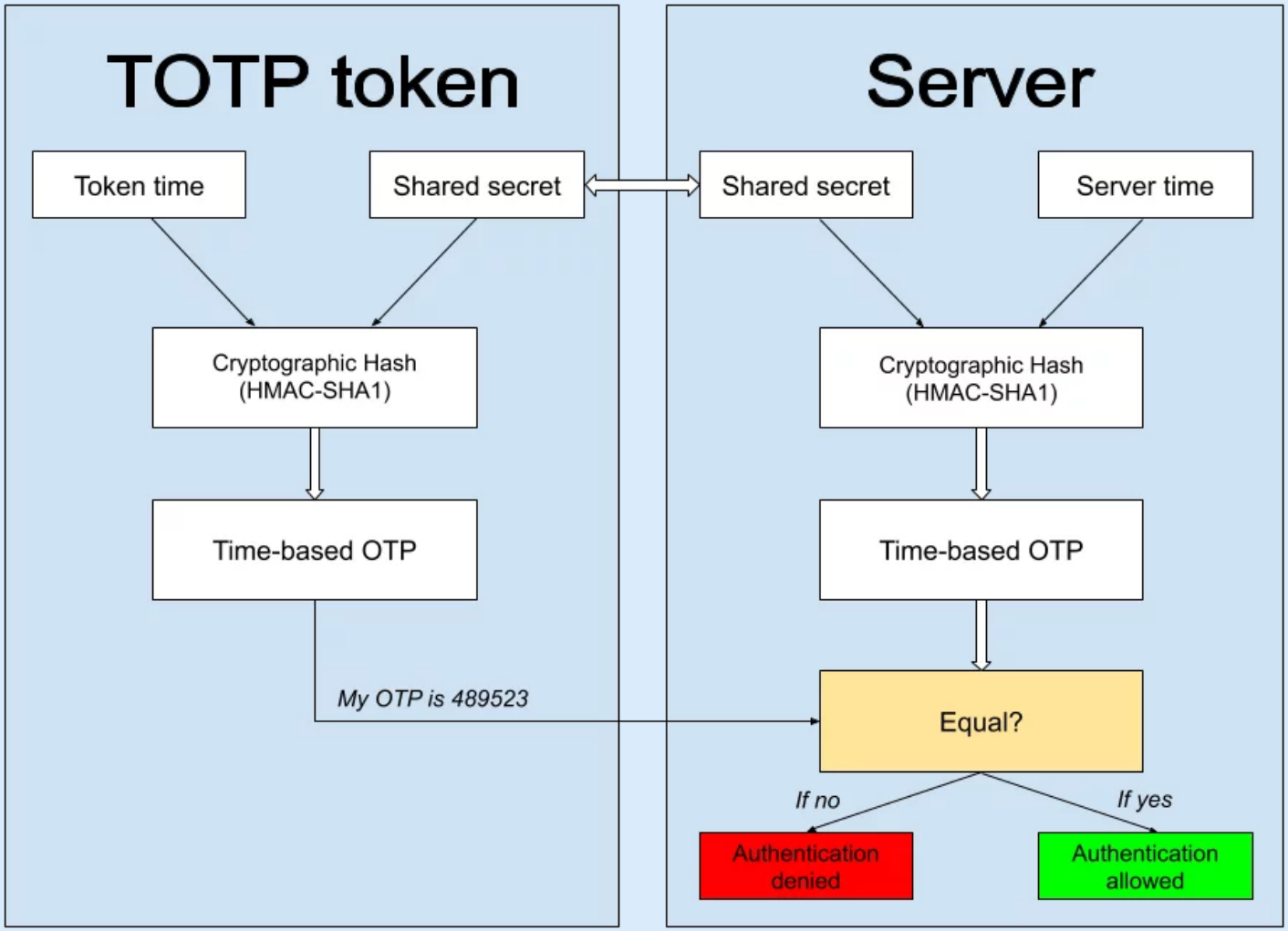
Time based one-time passcodes (TOTP)

- A secret value (seed) is securely handed over to client
- At authentication time, OTP is calculated at the client using **the current time** and seed
- A one-way function is used for OTP calculation (why?)



TOTP working





https://www.protectimus.com/blog/totp-algorithm-explained/

TOTP clock sync issues



- By its very nature, TOTP requires client and server clocks to be in close synchronization.
- In practice, real devices have a clock drift
  - small time keeping errors that accumulate over time
- For better user experience, most TOTP systems tolerate small clock drifts using these techniques

1) **Time step**: OTP is meant for a predefined interval
   - Typically, it is 30 sec. So the code from 05:21:00 to 05:21:29 will be same. At 05:21:30 a new code should be generated.
- **Acceptance window:** $OTP_{current} \pm 1$ step (or ± 2)
  - Server computes it for few extra time steps in past and future, and accepts if the code sent by client matches any of them

Downsides of OTPs

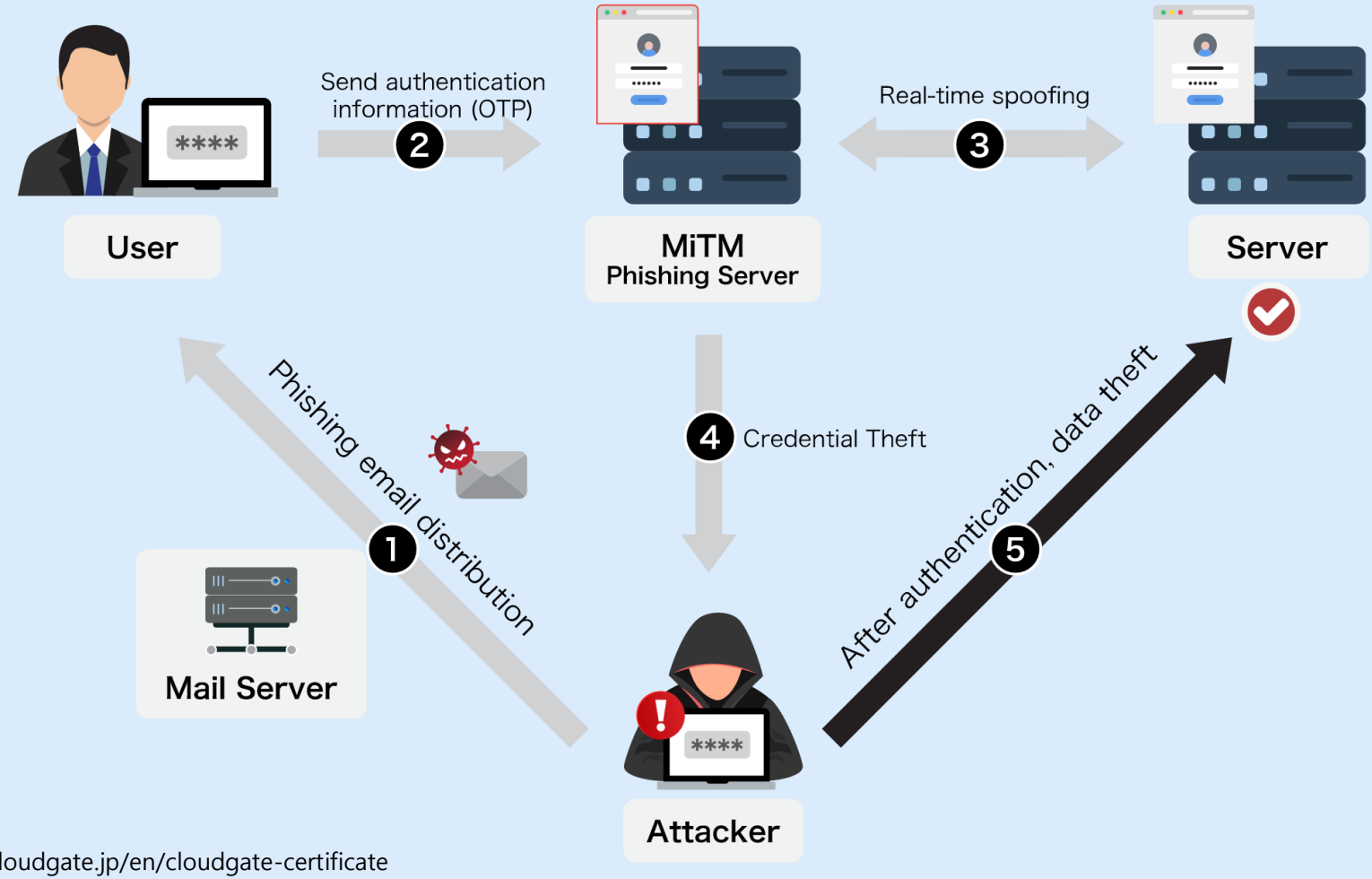


- OTPs have been very widely adopted as the second authentication factor.
- They are however, not without challenges, like:
- **Plaintext storage on server side:**
  - Hacking may result in all secret seeds stolen.
- **Vulnerable to phishing:**
  - Attackers can create a lookalike web page for TOTP entry, attract the user to this phishing page, capture the passcodes, and send to actual server in real time to gain access. Combined with acceptance-window issue mentioned earlier, this has turned out to be a very realistic attack vector.

Downsides of OTPs



A possible scenario of OTP being captured by an attacker and relayed to server in real time for identity spoofing



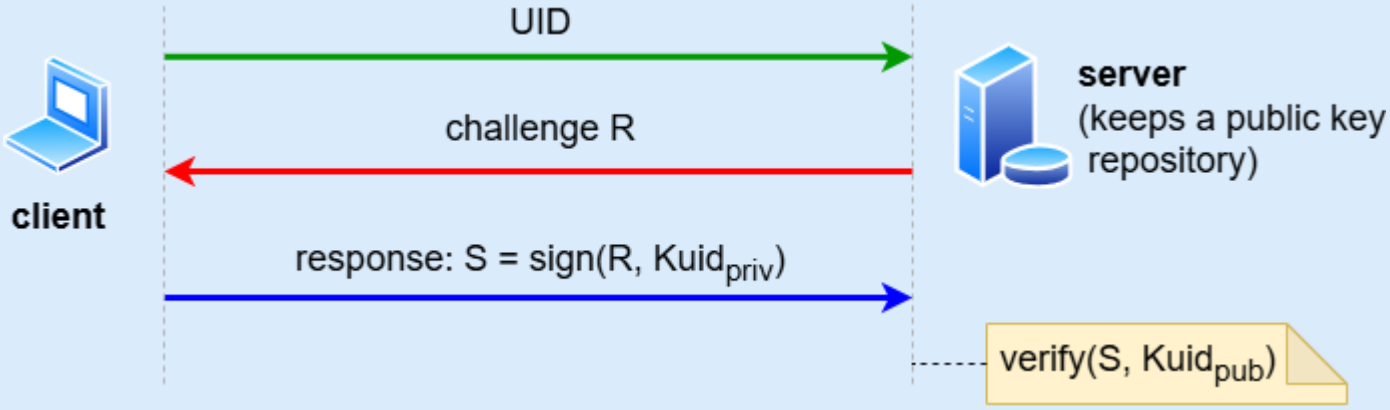
https://www.cloudgate.jp/en/cloudgate-certificate

Challenge-response authentication



Using Asymmetric Crypto

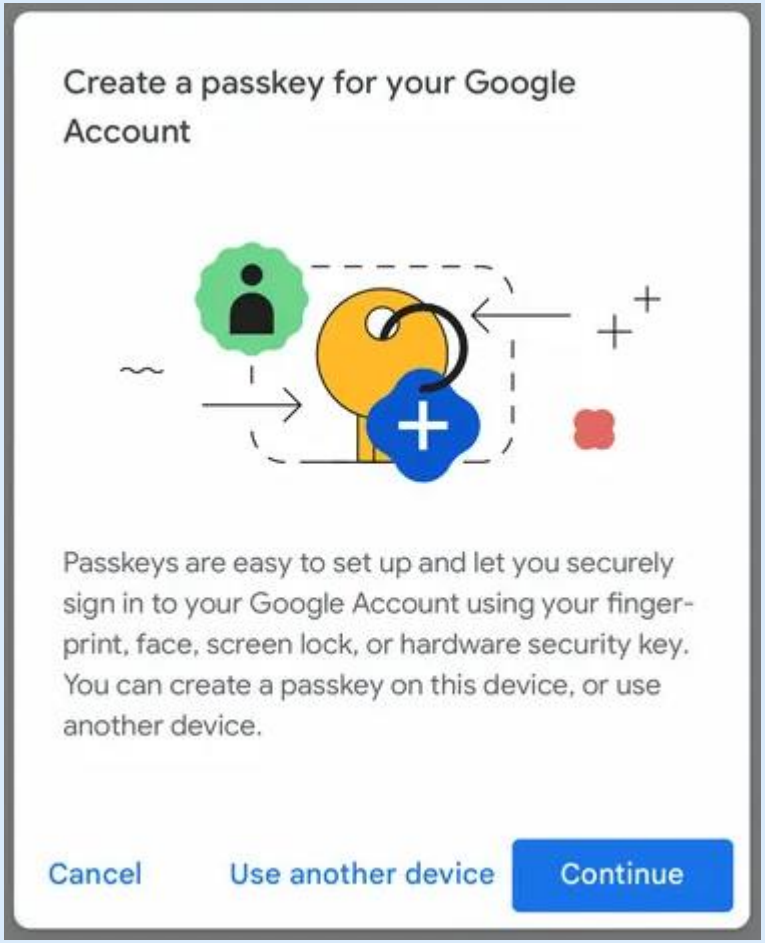
- server knows the public keys of all users
- server sends a challenge to user: a random nonce R
- user sends a response calculated using private key
  - e.g. encrypting or signing R with private key
- server verifies the response against user's public key



Passkeys



- A modern standard of authentication being increasingly adopted.
- It is based on the same idea of **challenge response authentication using public/private key pairs**.
- But further enhanced by associating each passkey with a specific website/ domain.



https://www.cnet.com/tech/services-and-software/ditch-your-password-set-up-a-passkey-for-your-google-account/

Passkeys



- To use passkeys, you need an **authenticator** software or dedicated hardware
  - e.g. Windows Hello, Apple Passwords, Yubikey device, or a third-party app (1Password, Bitwarden etc.)
- Authenticator jobs
  - create the keypair
  - securely hold the private keys
  - respond to a challenge (signing with private key)
  - perform **user verification** (like biometric check or device password) before responding to a challenge.
- The website stores the public key and uses it to verify the response.

Passkeys Strength



- What makes passkeys better than passwords and OTPs? **They are phishing resistant.**
- How?
  - Each passkey is scoped: it is associated to specific web domains.
    - A passkey created for google.com, will not work on a phishing website with a different domain.
  - The browser/authentication system enforces this restriction, preventing the passkey from being used for the wrong website.
- Because of their advantages and convenience of use, passkeys popularity is rising, it is meant to replace passwords.

Biometric authentication

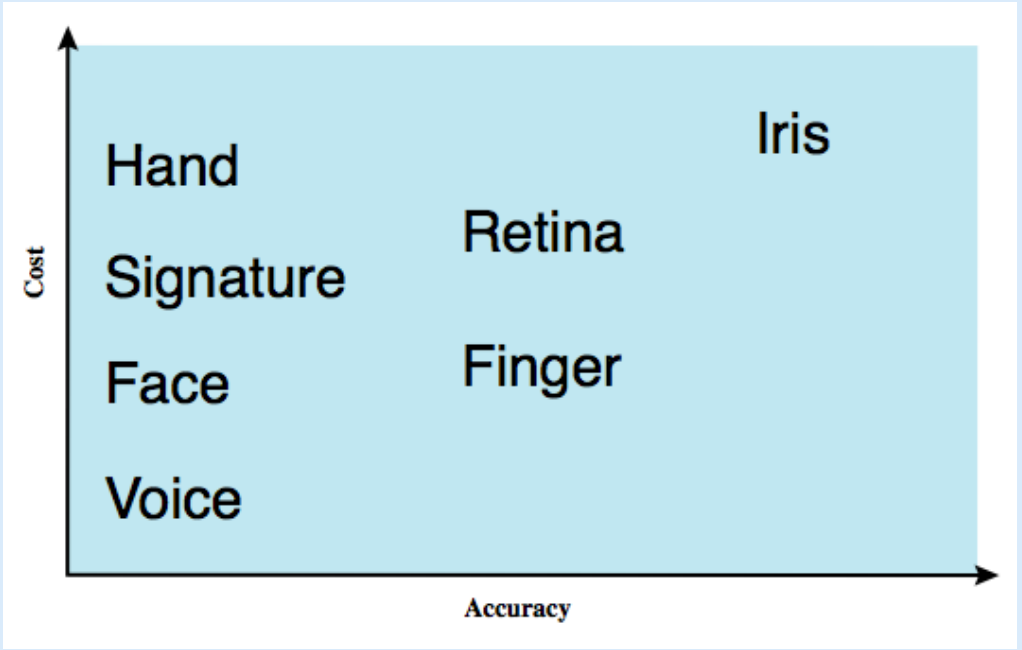
(someone you are)



Biometric Authentication



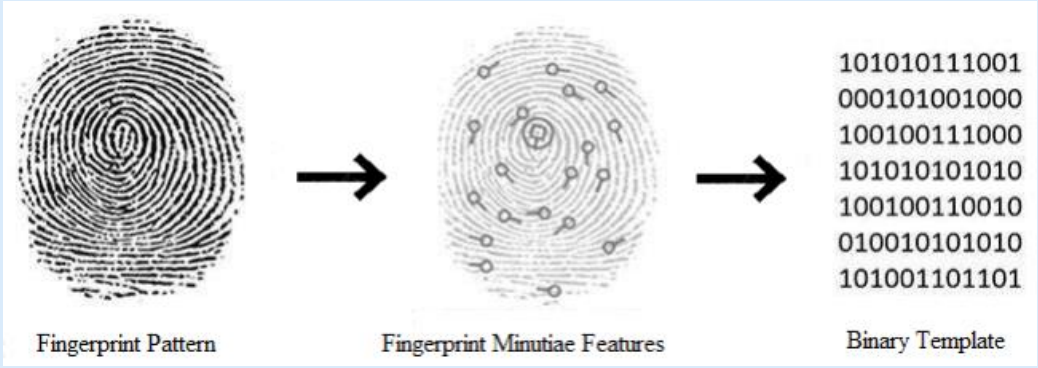
- Authenticate user based on one of their physiological characteristics:
  - facial features
  - fingerprint
  - hand geometry
  - palm vein pattern
  - retina vessel pattern
  - iris pattern
  - signature dynamics
  - voice



Features and Templates



- Specialized sensors are used to capture the biometric data.
- During enrollment, the raw data (image/audio) is not stored by itself. Instead, the system performs feature extraction to convert it into a numeric vector, called biometric **template**.
  - Template cannot easily be reversed back into the original biometric data, which is important for security and privacy.



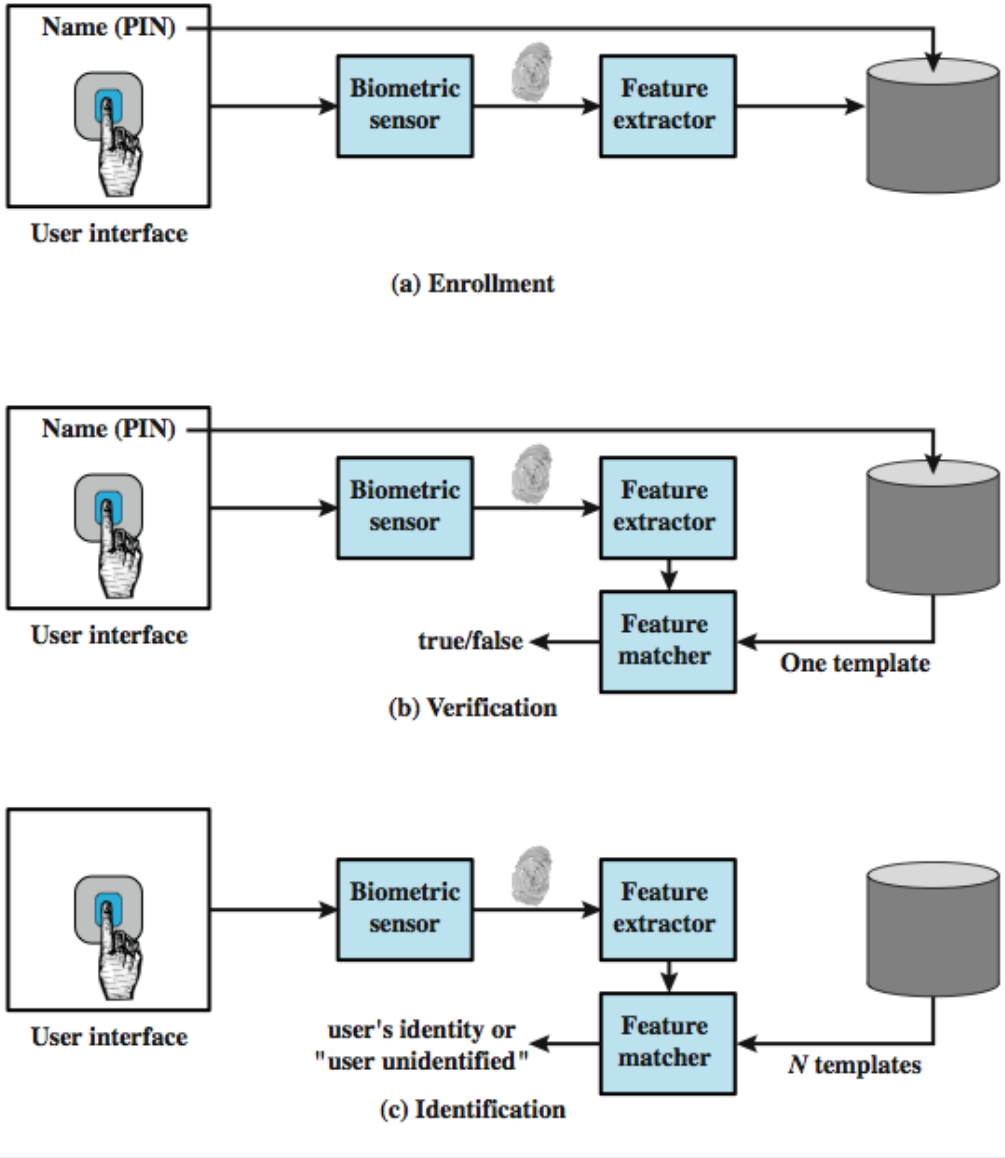
https://www.researchgate.net/publication/354801909_Study_on_Introducing_Biometric_Fingerprint_Authentication_in_Automated_Student_Attendance_System

Operation of a biometric system

Enrollment means registration of a user's template (unique trait)

Verification is when user provides their identity (username, PIN etc.) plus biometric data to get authenticated

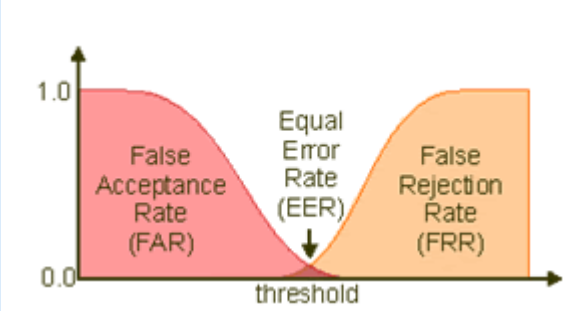
Identification is when only biometric data is provided, system compares it with all the stored templates



Template matching issue



- Due to sensor noise and variability in human usage, there won't be an exact match between any two biometric scan templates
  - So, system computes a **similarity score** (say 0 to 100). If the score is above some **threshold**, the user is accepted otherwise rejected
  - FAR = False Acceptance Rate
    FRR = False Rejection Rate



Using a higher threshold will reduce FAR (unauthorized users gaining access), at the cost of more genuine users getting blocked (high FRR)!

http://www.face-tek.com/e_bias_detail.php?BiasID=3

Other challenges in biometric auth



- Biological characteristics are variable due to aging, wounds, swelling, wetness etc.
  - voice altered due to emotion or injury: https://youtu.be/iYhpbph4sLc
  - retinal blood pattern altered due to alcohol or drug
- Persons with disability unable to use the system, so a fallback is needed
- Extra hardware & logistical costs of sensors
- Anonymous access becomes impossible

Centralized Multi-Service Authentication



Multi-Service Authentication



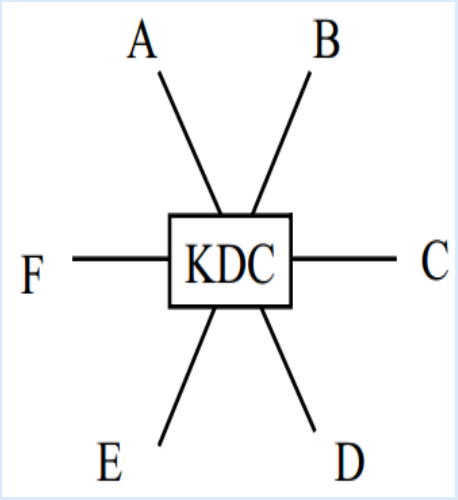
- A typical enterprise environment has several application servers
  - e.g. sales management, knowledgebase, wiki, accounts, HR management, technical support
- It will be too complex (and insecure) to store separate user credentials or keys on each application server
- Solution: designate a **single authentication server** for all auth requests
- The solution can be implemented using either symmetric or asymmetric crypto.

Key Distribution Center (KDC)



KDC is a solution for multi-party secure communication using **symmetric cryptography**

- Each node is configured with a permanent secret key, shared with KDC.
- Only KDC has the keys of all users.
- For an n-user system, KDC has to maintain n keys.



Key Distribution Center (KDC)



Suppose user A wants to initiate a conversation with user B.

- A sends a request to KDC: "I want to talk to B"
- KDC generates a random session key K_{AB} and creates two packages
  - Package1 $\{KAB\}KA$: session key encrypted with A's secret
  - Package2 $\{KAB\}K_B$: session key encrypted with B's secret
- KDC sends both packages back to A
- A opens package1 and gets the session key
- A forwards package2 to B, who decrypts it and gets the same session key
- Now A and B can have a conversation, secured with K_{AB}
- Once the session is finished, both can discard K_{AB}

Kerberos





- Network authentication protocol
- Based on idea of using symmetric crypto
  - Maintain a KDC
- Developed at MIT for project Athena

- Named after Greek mythological character “Cerberus” – the three headed dog
- Used by popular operating systems and servers
- Protects against eavesdropping and replay attacks

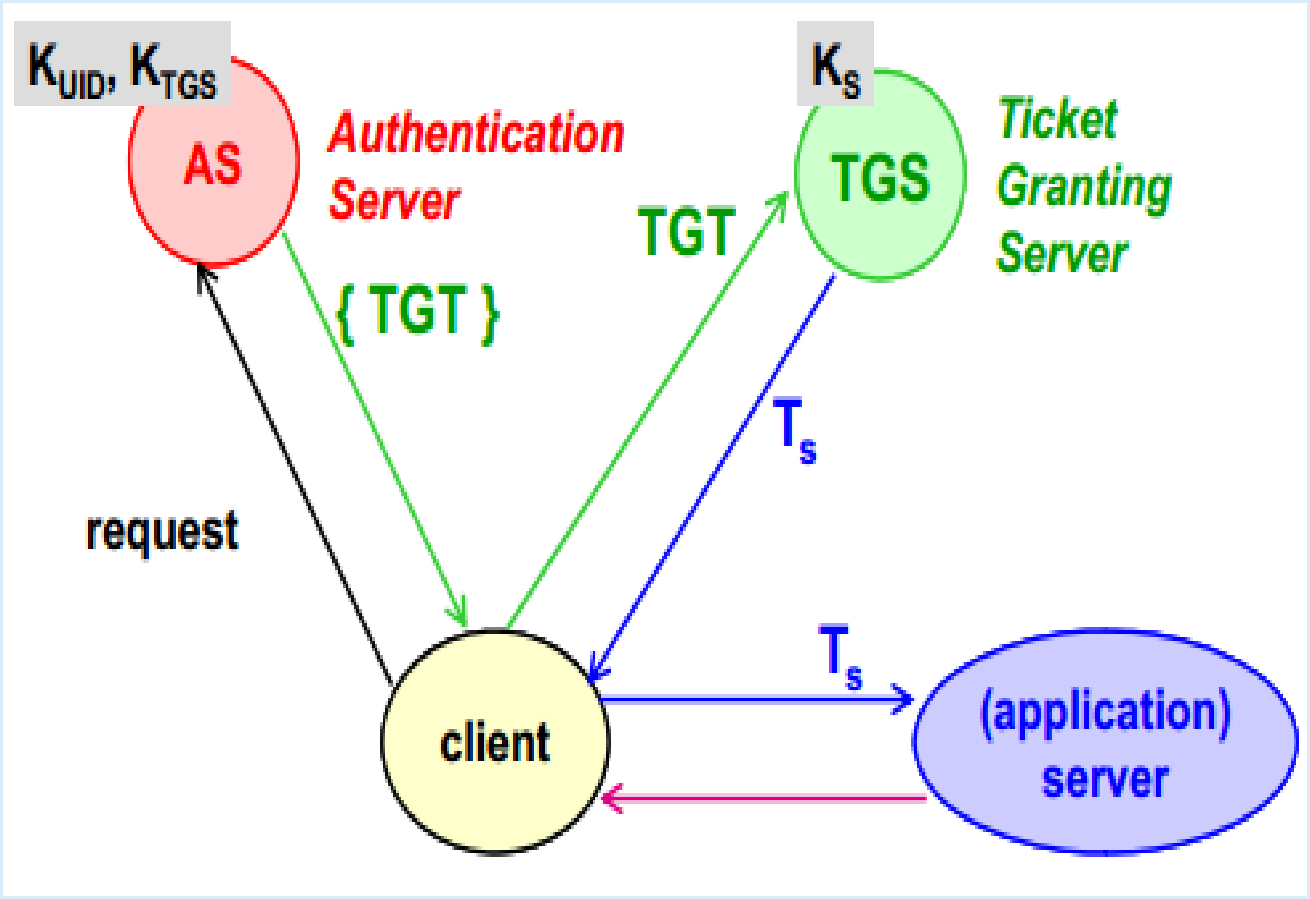
Kerberos overview



- A central Key Distribution Center (KDC) is maintained, which further contains two logical components
  - Authentication server that authenticates a user
  - Ticket Granting Server (TGS), that grants ticket to the user, for a specific service in the network
- Application Server provides the service to the user
- The client, the KDC and the Application server are the 3 heads in Kerberos

Kerberos High Level Working





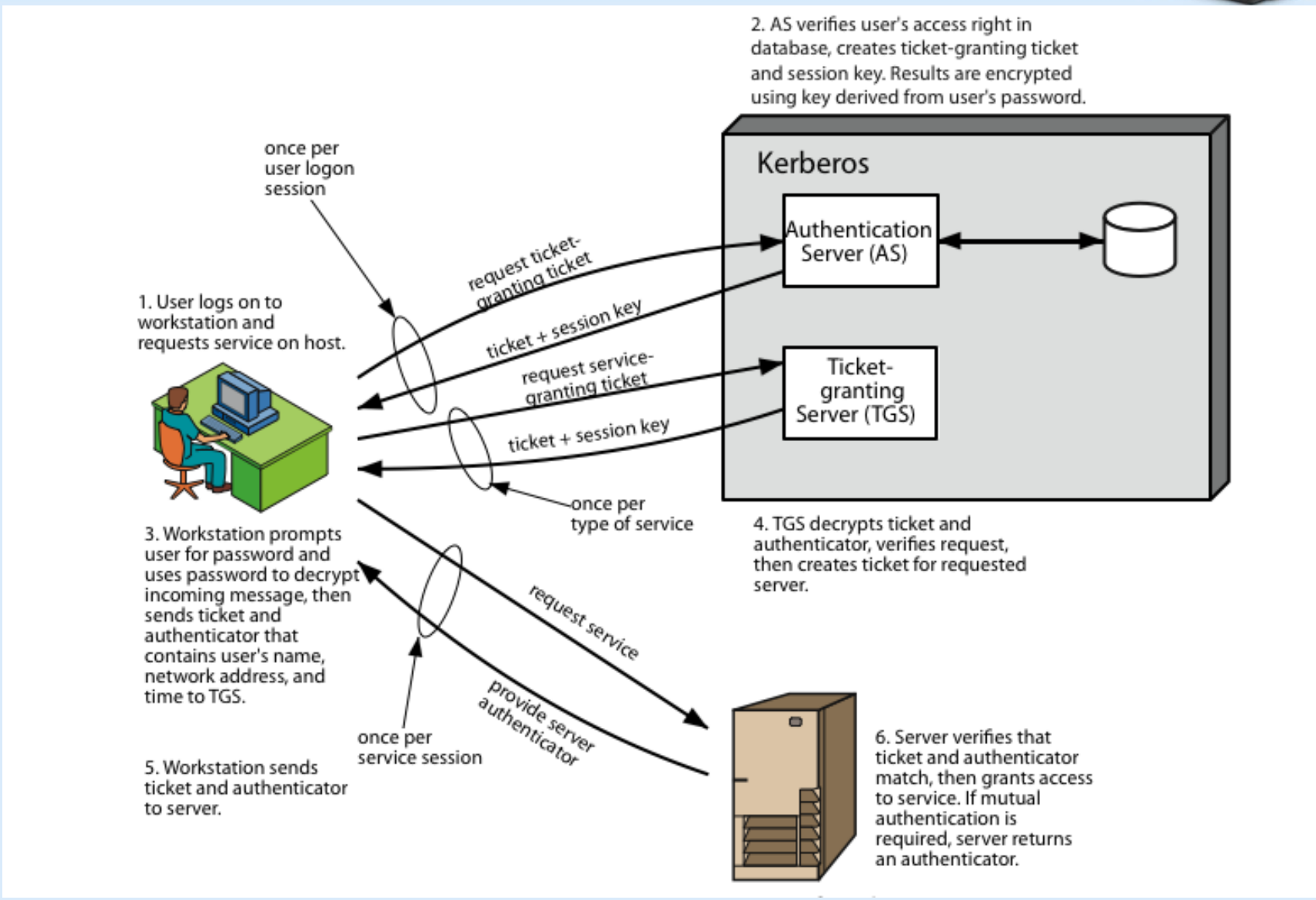
TGT = ticket granting ticket T_s = service ticket

K_{UID} = secret key of user, K_{TGS} = secret key of TGS; both pre-shared with AS

K_S = secret key of application server, pre-shared with TGS

Kerberos Protocol



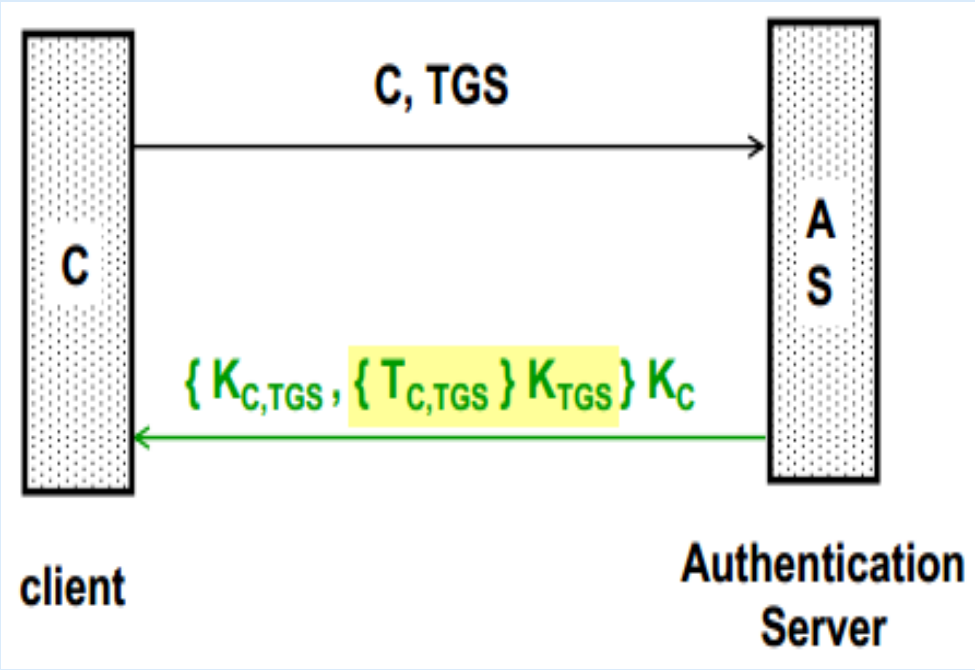


1-2. Client Authentication (Issue TGT)



Request

- client send their id C, and ask for a ticket that will help them connect to ticket granting server TGS
- this message is in plaintext



Response contains

- $K_{C,TGS}$: the session key to connect to ticket granting server TGS
- $T_{C,TGS}$: the ticket-granting ticket. It is encrypted with K_{TGS} (the secret key of TGS), not known to client

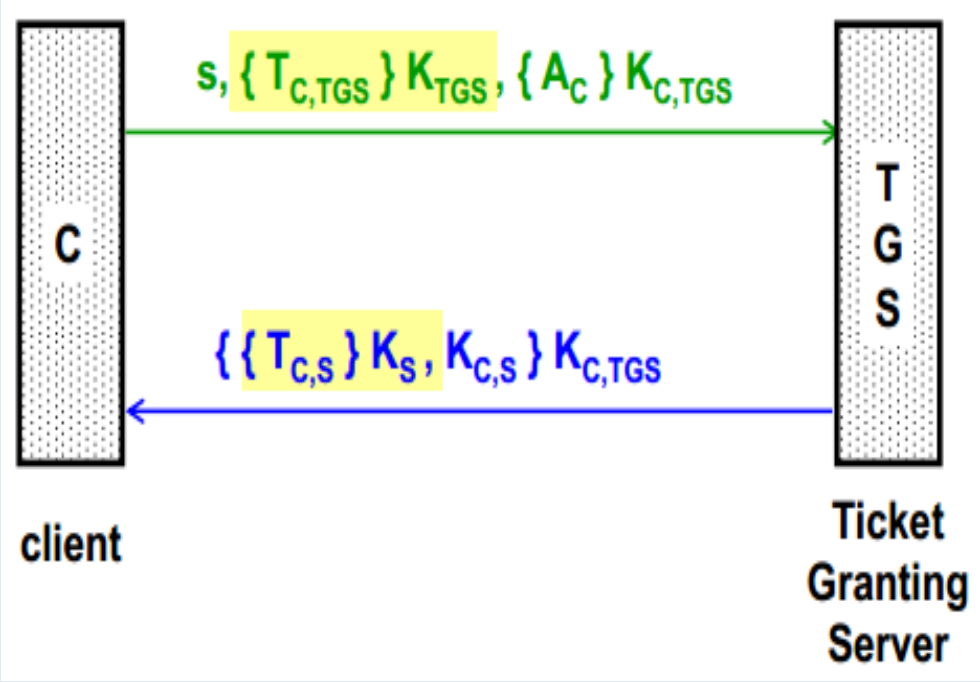
Whole response is encrypted with the secret key of client K_C (derived from their password)

3-4. Client Authorization (Issue Service Ticket)



Request

- s : id of the service that client wants to use
- Encrypted TGT (yellow highlighted)
- A_C : an authenticator message from client, proving his identity to TGS. It is encrypted by this session's key $K_{C,TGS}$



Response

- $T_{C,S}$: service ticket, encrypted with key of application server K_S (not known to client)
- $K_{C,S}$: A session key for use between client and application server

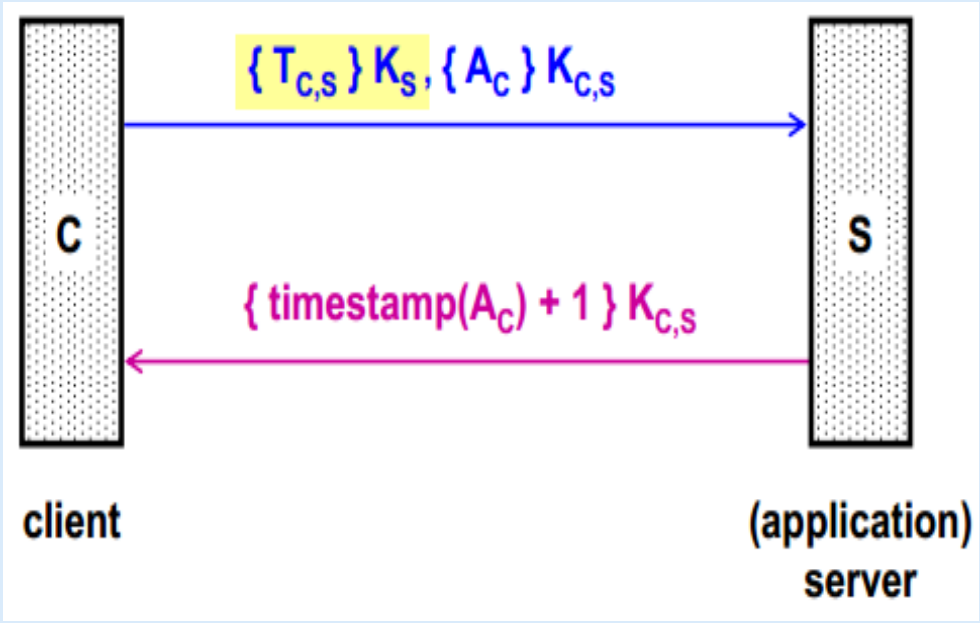
Whole message is encrypted by this session's key $K_{C,TGS}$

5-6. Client Service Request



Request

- the encrypted service ticket (yellow highlighted)
- A_C : an authenticator message containing client ID and timestamp. It is encrypted by this session's key $K_{C,S}$.



Response

- Timestamp in client's message plus 1, encrypted by this session's key $K_{C,S}$. It is a confirmation to client that server is ready to serve it.

References



- Stallings SPP chap 3, 23
- Demonstration of TOTP and passkey-based 2FA https://www.cryptool.org/en/cto/2fa/
- Kerberos on Computerphile: https://youtu.be/qW361k3-BtU
- Kerberos message exchange in full detail https://youtu.be/5N242XcKAsM